

**INTELLIGENCE BASED GENERATIVE MODEL FOR
PROGRAMMING LANGUAGE**

A Project Report submitted in partial fulfillment of the
requirements for the award of the degree of
BACHELOR OF TECHNOLOGY
IN
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

SUBMITTED BY:

**AGATHEESWARAN A (8208E22ASR008),
HASEEBMOHAMED H (8208E22ASR022),
PRAVEENKUMAR S (8208E22ASR040).**

UNDER THE GUIDANCE OF:

**Mr. R. GOBINATH, M.E., (Ph.D.)
ASSISTANT PROFESSOR / AI & DS**



**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA
SCIENCE**

**E.G.S. PILLAY ENGINEERING COLLEGE (AUTONOMOUS)
(Accredited by NBA & NAAC with 'A++' Grade)
NAGAPATTINAM – 611 002**

SUBMISSION DATE: APRIL 2026

ABSTRACT

The rapid proliferation of diverse programming languages and the increasing complexity of software systems have created a significant cognitive burden for modern developers. This research addresses the challenge of "tool fragmentation" by developing a **Multi-tasking AI Code Assistant**, a unified web-based ecosystem designed to streamline the Software Development Life Cycle (SDLC). The core of the system is powered by **CodeGen-2B-Multi**, a 2-billion parameter large language model (LLM) developed by Salesforce, which is optimized for multi-step program synthesis across various high-level languages including Python, Java, and C++.

The methodology involved a multi-tiered architectural approach. A high-performance, asynchronous backend was engineered using the **FastAPI** framework to manage intensive model inference tasks, while a responsive, "Vanilla" JavaScript and CSS3 frontend was developed to ensure a lightweight and accessible user experience. The system's intelligence was realized through advanced **In-Context Learning** paradigms, specifically **Zero-Shot** and **One-Shot learning**, allowing the assistant to perform four distinct operations: automated code generation, syntax and logical debugging, language-to-language translation, and semantic code explanation.

To optimize the system for resource-constrained cloud environments like **Render**, several technical strategies were implemented, including **FP16 quantization**, **KV-caching**, and manual garbage collection to manage memory overhead. Quantitative evaluation demonstrated a robust performance profile, with an average inference latency of **1.5 to 4.0 seconds** and a success rate exceeding **85%** on common debugging and translation benchmarks. Qualitative analysis further confirmed the model's ability to provide human-readable explanations that bridge the gap between technical logic and conceptual understanding.

This project contributes to the field of Applied Artificial Intelligence by providing a scalable blueprint for deploying LLMs in production-ready environments. The results suggest that integrated AI tools significantly enhance developer productivity and provide a critical educational resource for novice programmers. Future research will explore the integration of larger model architectures and the implementation of real-time IDE plugins to further embed AI assistance into the native developer workflow.

Keywords: *Artificial Intelligence, Large Language Models (LLMs), CodeGen, FastAPI, Software Engineering, Transformer Architecture, Code Debugging, Machine Translation.*

ACKNOWLEDGMENTS

I would like to express my deepest gratitude to **EGS Pillay Engineering College** for providing the infrastructure and academic environment that made this research possible. I am profoundly indebted to our Head of Department, **Dr. J. Amutha (Assistant Professor)**, for her unwavering support and for fostering a culture of innovation within the **AI&DS Department**. Special recognition is due to my supervisor and coordinator, **Mr. R. Gobinath (Assistant Professor)**, whose technical expertise, patience, and meticulous guidance were instrumental in shaping the "Intelligence Based Generative Model for Programming Language." His insights into large language models and backend architecture provided the foundation for our success.

I also wish to extend my sincere thanks to the dedicated faculty members of the AI&DS Department, including **Ms. Rahapriya, Ms. Divyabharathi, and Ms. Akilandeswari (Assistant Professor)**, whose scholarly advice and administrative assistance were vital during the various stages of the project. Furthermore, We Are grateful to be teammates—**Agatheeswaran A, PraveenKumar S, and HaseebMohamed H**—whose collaborative spirit, technical contributions, and tireless efforts turned our shared vision into a tangible accomplishment. Finally, I thank all the department staff and my peers for their constant encouragement and for providing the collaborative atmosphere required to complete this thesis.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
CHAPTER 1	INTRODUCTION	1
CHAPTER 2	LITERATURE REVIEW	12
CHAPTER 3	THEORETICAL BACKGROUND	22
CHAPTER 4	SYSTEM DESIGN AND ARCHITECTURE	33
CHAPTER 5	SYSTEM IMPLEMENTATION AND DOCUMENTATION	43
CHAPTER 6	CONCLUSION	55
CHAPTER 7	FUTURE WORK AND RECOMMENDATIONS	60
CHAPTER 8	REFERENCES (BIBLIOGRAPHY)	62
CHAPTER 9	APPENDICES	63

TABLE CONTENTS OF TABLE

TABLE	TITLE	PAGE NO
TABLE 1.1	CodeGen vs. GPT-3.5 vs. GPT-4	3-4
TABLE 1.2	Human Developer VS AI	5-6
TABLE 2.1	N-gram vs. Transformer	14-15
TABLE 2.2	Parameter & Dataset Comparison	16
TABLE 2.3	CodeGen Model Parameter	20
TABLE 2.4	Model Selection	21
TABLE 3.1	Zero-Shot Learning vs One-Shot Learning	29-30
TABLE 4.1	Feature Selection	47
TABLE 5.1	Configuration Usage	59
TABLE 5.2	Ratio Analysis	60
TABLE 6.1	Accomplishment	66-67

TABLE OF FIGURES

Figure	TILTLE	PageNO
Figure 1.1	Waves of AI Coding	1
Figure 3.1	Transformer	25
Figure 3.2	CodeGen-2B-Multi	26
Figure 4.1	Model Architecture	39
Figure 4.2	Example JSON Entry:	41
Figure 4.3	USER INTERFACE DESIGN	42
Figure 9.1	Sample Output	72
Figure 9.2	Journal Publication	73-74